

Chương 12: Mô hình tùy chỉnh và Huấn luyện với TensorFlow

Giảng viên: TS. Trần Thành Thắng
Đại học Đông Á
Năm học 2024-2025

Gemini Notebook

Nội dung chương



1. Giới thiệu TensorFlow & Kiến trúc
2. Sử dụng TensorFlow như NumPy
3. Tùy chỉnh mô hình và Thành phần (Custom Components)
4. Hàm và biểu đồ TensorFlow (AutoGraph)
5. Vòng lặp Huấn luyện Tùy chỉnh

Gemini Notebook

Giới thiệu nhanh về TensorFlow



Thư viện mã nguồn mở cho học máy quy mô lớn và học sâu. Cốt lõi tương tự NumPy, nhưng thao tác với **Tensor** thay vì mảng.



Tính toán phân tán hàng ngàn thiết bị. Hỗ trợ phần cứng tăng tốc mạnh mẽ: **GPU** (Graphics Processing Unit) và **TPU** (Tensor Processing Unit).



Chu vện Tự dịch biểu thức tính toán Python tiành **TPU** và Tensor Processing Unit.



Tự động biên dịch biểu thức tính toán Python thành **Biểu đồ (Graph)** C++ hiệu suất cao.

Gemini Notebook

Hệ sinh thái TensorFlow



TensorBoard

Trực quan hóa dữ liệu, đồ thị mô hình và phân tích quá trình huấn luyện.



TensorFlow Hub

Nơi tải xuống và sử dụng lại các mô hình đã huấn luyện (Pretrained models) giúp đẩy nhanh tiến độ.



TensorFlow Extended (TFX)

Bộ công cụ xây dựng quy trình ML toàn diện và triển khai mô hình lên môi trường sản xuất (Production).



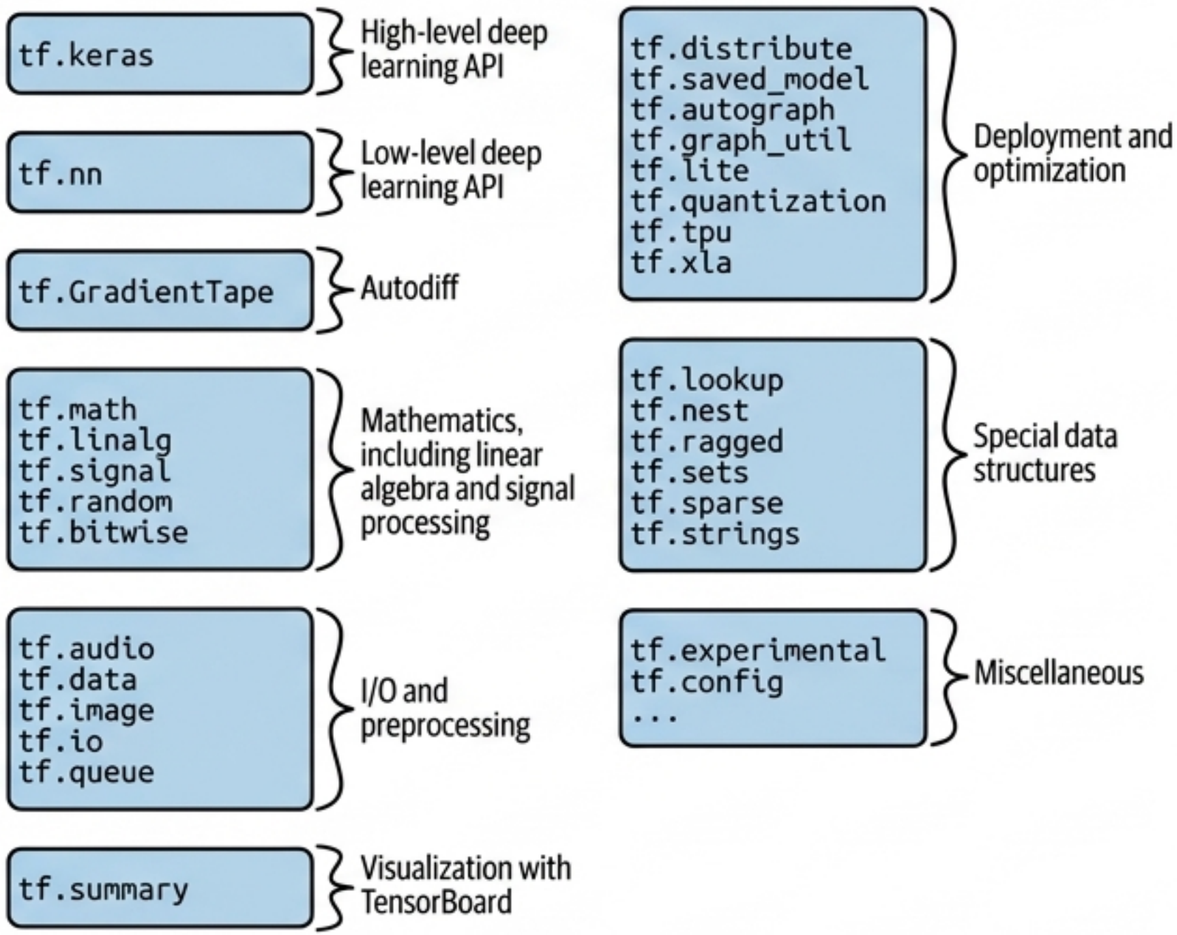
TF Lite & TF.js

Triển khai mô hình trên di động (Android, iOS), phần cứng nhúng (Raspberry Pi) và ngay trên trình duyệt Web bằng JavaScript.

Gemini Notebook

Tổng quan về API Python của TensorFlow (1/2)

- API Python thiết kế phân tầng từ thấp đến cao.
- **Cấp thấp (Nền tảng):** Phép toán trên Tensor, Biến số (Variables), Tự động vi phân (**tf.GradientTape**).
- **Cấp cao:** Keras (**tf.keras**) cung cấp API trực quan xây dựng lớp (layers) và mô hình (models).
- **Dữ liệu:** API dữ liệu (**tf.data**) nạp, xử lý trước và cung cấp dữ liệu huấn luyện hiệu quả.



Hình 12-1: Tổng quan về API Python của

Gemini Notebook

Tổng quan về API Python của TensorFlow (2/2)



Người học mới (Keras)

Sự trừu tượng hóa mức độ phức tạp giúp tiếp cận nhanh chóng thông qua **Keras**.



Nhà nghiên cứu (Low-level API)

Viết các thuật toán huấn luyện phi tiêu chuẩn (như trong **GANs** hay **Meta-learning**) bằng các **API cấp thấp**.

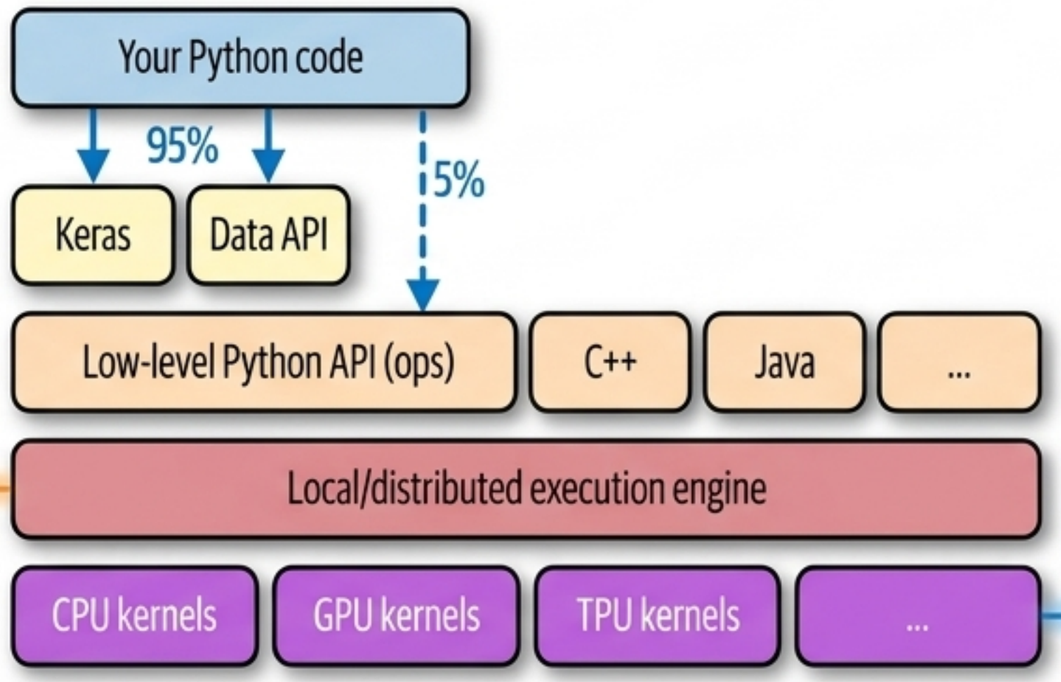
Tổng kết: Hình 12-1 tóm tắt bao quát các module quan trọng. Chúng ta sẽ khám phá từ **Tensor cơ bản** đến **tùy chỉnh lớp/mô hình** qua API này.

Gemini Notebook

Kiến trúc của TensorFlow (1/2)

Hình 12-2 minh họa kiến trúc bên trong của TensorFlow.

Execution Engine (Trung tâm): Lớp trung tâm thực hiện các biểu đồ (Graphs) bằng C++ (mã nguồn lõi tối ưu cực cao).



Đa nền tảng (Bên dưới): Khả năng chạy đồng thời (multi-threading) trên nhiều loại phần cứng (CPU, GPU, TPU) và qua mạng (Android, iOS, Cloud).

Hình 12-2: Kiến trúc của TensorFlow

Gemini Notebook

Kiến trúc của TensorFlow (2/2)



Đa ngôn ngữ

API giao diện: Python (phổ biến nhất), C++, Java, Go, Swift. Tương tác chủ yếu qua **Keras** (High-level API).



Lớp XLA (Accelerated Linear Algebra)

Trình biên dịch mạnh mẽ, tối ưu hóa **Graph**. Kết hợp toán tử (kernel fusion) để tăng tốc tối đa phần cứng.



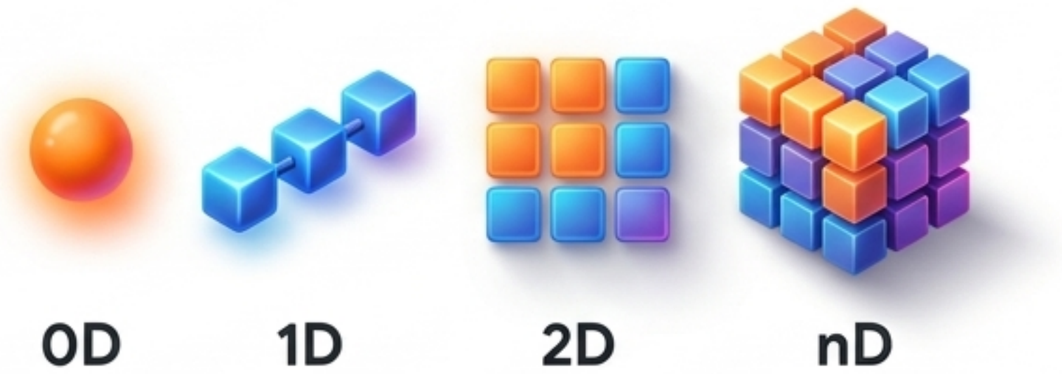
TF Distributed Execution

Phân phối trực tiếp tính toán trên nhiều máy vật lý (**Parameter Server**, **MirroredStrategy**).

Gemini Notebook

Tensor và Phép toán (Tensors and Operations)

- **Định nghĩa:** **Tensor** là cáp số liệu đa chiều kế thừa từ Toán học: vô hướng (scalar - 0D), vectơ (1D), ma trận (2D), không gian đa chiều (nD).
- **Khởi tạo:** Cách dễ nhất là dùng **tf.constant()**.



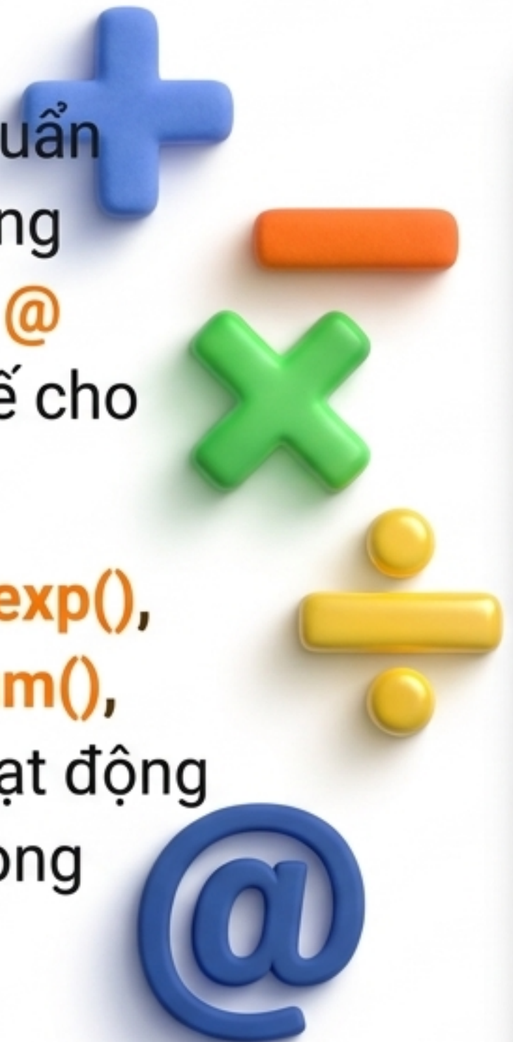

```
import tensorflow as tf

# Tao tensor tu ma tran 2 chieu (2D)
t = tf.constant([[1., 2., 3.], [4., 5., 6.]])
print(t.shape) # In ra: TensorShape([2, 3])
print(t.dtype) # In ra: <dtype: 'float32'>
```

Gemini Notebook

Các hàm toán học cơ bản

- Tính năng: Toán tử chuẩn **Python** được nạp chồng (**overloaded**). Ký hiệu **@** **(Python 3.5+)** thay thế cho **tf.matmul**.
- Các hàm phổ biến: **tf.exp()**, **tf.sqrt()**, **tf.reduce_sum()**, **tf.reduce_mean()**. Hoạt động trực tiếp không cần vòng lặp for.



```
# Tinh toan co ban bang TF operations
t1 = t + 10 # Phép cong (hoạt dọng broadcasting)
t2 = tf.square(t) # Bình phương tưng phan tư (element-wise)
t3 = t @ tf.transpose(t) # Nhan ma tran (Matrix multiplication)
```

Gemini Notebook

Tensors và NumPy: Tương tác qua lại

Đặc tính: **Tương tác thân thiện, liền mạch. Chuyển đổi trực tiếp** và **tính toán kết hợp (Cross-computation)** dễ dàng.



Terminal Box

```python
import numpy as np


# Chuyen Numpy -> Tensor
a = np.array([2., 4., 5.])
t_a = tf.constant(a)

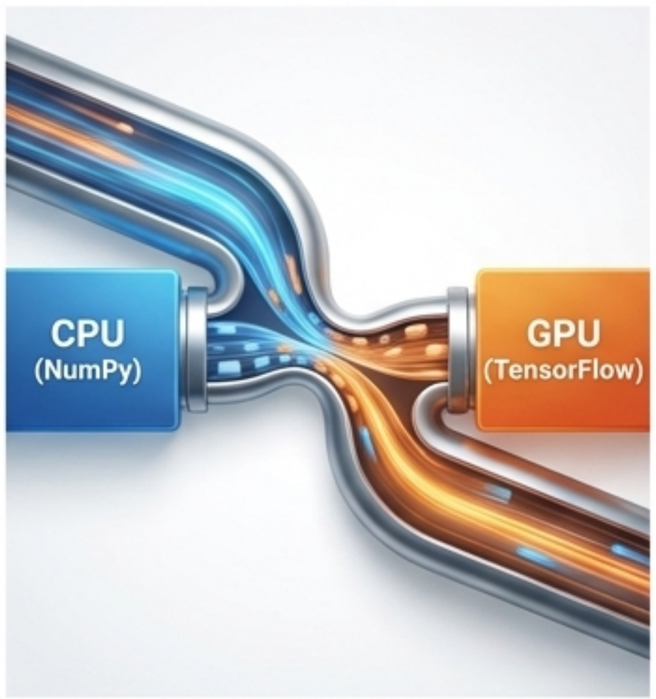
# Chuyen Tensor -> Numpy
a_back = t_a.numpy() # Hoac dung np.array(t_a)

# Tinh toan ket hợp (Cross-computation)
t_b = tf.square(a) # TF chap nhan Numpy array
a_b = np.square(t_a) # Numpy chap nhan TF Tensor
```

Gemini Notebook

Lưu ý khi dùng chung Numpy và TensorFlow





- **Nghẽn cổ chai (Bottleneck):** **NumPy** chỉ chạy trên **CPU**. Nếu gọi hàm NumPy cho chạy trên CPU. Nếu gọi hàm NumPy cho **Tensor** đang ở **GPU**, TensorFlow phải sao chép dữ liệu từ **GPU** về CPU → **Chi phí truyền tải lớn.**
- **Giải pháp:** Ưu tiên dùng hàm **TensorFlow** để tận dụng tốc độ **GPU**.
- **Khác biệt kiểu dữ liệu mặc định:**
  - **NumPy:** Dùng **float64** (độ chính xác cao).
  - **TensorFlow:** Dùng **float32** hoặc **bfloat16** (mạng Neural ưu tiên hiệu suất).

Gemini Notebook

Chuyển đổi kiểu dữ liệu (Type Conversions)

Nguyên tắc: **TensorFlow không tự động ép kiểu** như Python/NumPy. Ép kiểu giữa **float32/float64** hoặc **float32/int32** có **chi phí cao**.

Lỗi TypeError



Problem & Solution

tf.cast



```
# Kieu int32 khong the cong voi float32 ngam dinh:
# tf.constant(2.0) + tf.constant(40) # --> Loi TypeError

# Can phai ep kieu chu dong va ro rang bang tf.cast:
t2 = tf.constant(40., dtype=tf.float64)

# tf.cast(t2, tf.float32) ha do chinh xac xuong float32
result = tf.constant(2.0) + tf.cast(t2, tf.float32)
```

Gemini Notebook

Biến số (Variables)



Vấn đề: `tf.constant()` có tính **immutable** (không thể thay đổi). Nhưng **mạng nơ-ron** cần liên tục cập nhật trọng số (weights) và độ lệch (biases) qua mỗi chu kỳ học (epoch).



Giải pháp: Sử dụng `tf.Variable`.

Terminal Box

```
v = tf.Variable([1.,2., 3.], [4., 5., 6.]])

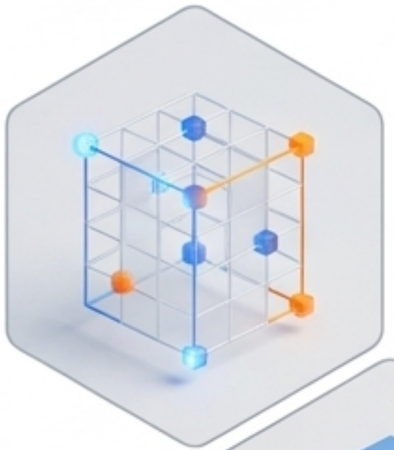
# Cap nhat toan bo bien (dung phuong thuc assign)
v.assign(2 * v)

# Cap nhat vao dung 1 vi tri cu the (nhu numpy arrays)
v[0, 1].assign(42)

# Cap nhat toan cục bang phep cong (+)
v.assign_add([1., 1., 1.], [1., 1., 1.]])
```

Gemini Notebook

Các cấu trúc dữ liệu nâng cao khác



Sparse tensors (tf.SparseTensor): Tối ưu **bộ nhớ** cho ma trận chứa phần lớn **số 0**. Lưu 3 mảng: **indices**, **values**, **dense_shape**.



Ragged tensors (tf.RaggedTensor): Mảng **bất đối xứng** (VD: các chuỗi con độ dài không bằng nhau).



Tensor arrays (tf.TensorArray): Danh sách Tensors cho phép **append động**. Hữu ích cho mạng tuần tự **RNN**.



String tensors: Lưu trữ kiểu chữ, chuỗi **UTF-8** hoặc **byte arrays**, cần thiết cho **NLP**.



Sets: Tập hợp Toán học dưới dạng **Sparse tensor** qua tf.sets.

Gemini Notebook